



**INSTITUTO
FEDERAL**
Santa Catarina

Expressões, restrições e transações SQL

Instituto Federal de Santa Catarina

Análise e Desenvolvimento de Sistemas
Banco de dados

Prof. Ana Luiza Scharf
ana.scharf@ifsc.edu.br



**INSTITUTO
FEDERAL**
Santa Catarina

Expressões no banco de dados

Por que usar?

Expressões

- Permitem **manipular e transformar dados** diretamente no SQL.
 - em inves de mostrar só o salário do funcionário, você pode mostrar o salário com o bônus já aplicado.
- Tornam as consultas **mais dinâmicas e inteligentes**.
- Possibilitam a criação de **colunas derivadas** e **valores calculados**.
- Facilitam a aplicação de **regras de negócio** no próprio banco de dados.
- Ajudam na **construção de relatórios e análises** sem alterar os dados originais

Expressões

- Permitem **manipular e transformar dados** diretamente no SQL.
- Tornam as consultas **mais dinâmicas e inteligentes**.
 - em vez de mostrar só a nota do aluno, você pode indicar se ele está aprovado, em recuperação ou reprovado.

Expressões

- Permitem **manipular e transformar dados** diretamente no SQL.
- Tornam as consultas **mais dinâmicas e inteligentes**.
- Possibilitam a criação de **colunas derivadas e valores calculados**.
 - em vez de exibir “1” ou “0” numa coluna de status, você pode mostrar “Ativo” ou “Inativo”.

Expressões

- Permitem **manipular e transformar dados** diretamente no SQL.
- Tornam as consultas **mais dinâmicas e inteligentes**.
- Possibilitam a criação de **colunas derivadas** e **valores calculados**.
- Facilitam a aplicação de **regras de negócio** no próprio banco de dados.
 - a data pode estar salva como “2025-05-13”, mas você pode exibí-la como “13/05/2025” .

Expressões

- Permitem **manipular e transformar dados** diretamente no SQL.
- Tornam as consultas **mais dinâmicas e inteligentes**.
- Possibilitam a criação de **colunas derivadas** e **valores calculados**.
- Facilitam a aplicação de **regras de negócio** no próprio banco de dados.
- Ajudam na **construção de relatórios e análises** sem alterar os dados originais.
 - ao invés de mostrar a data de nascimento, você pode calcular e exibir direto a idade da pessoa.

Benefícios práticos

- Redução da dependência de código externo;

.

Benefícios práticos

- Redução da dependência de código externo;
- Melhor desempenho em consultas complexas;

Benefícios práticos

- Redução da dependência de código externo;
- Melhor desempenho em consultas complexas;
- Maior clareza e controle sobre a apresentação dos dados.

Relembrando ...

- **Aritméticas**, como somar, subtrair ou calcular porcentagens diretamente nas colunas (ex: calcular salário com bônus ou total de vendas).
 - Exemplo: calcular o salário com bônus ($\text{salário} * 1.1$) ou o total de vendas ($\text{quantidade} * \text{preco_unitario}$).

Relembrando ...

- **Aritméticas**, como somar, subtrair ou calcular porcentagens diretamente nas colunas (ex: calcular salário com bônus ou total de vendas).
- **Funções de texto**, para juntar nomes, converter para maiúsculas ou minúsculas, extrair partes de palavras, etc.
 - Exemplos:
 - • **CONCAT()** – para juntar nome e sobrenome em uma única coluna;
 - • **UPPER()** / **LOWER()** – para deixar o texto todo em maiúsculas ou minúsculas;

Relembrando ...

- **Aritméticas....**
- **Funções de texto, ...**
- **Funções de data**, para formatar datas, calcular idades ou descobrir o dia da semana de um evento.
 - Exemplos:
 - • **DATE_FORMAT()** (no MySQL) – para mudar o formato da data e deixá-la mais legível;
 - • **YEAR()** ou **DATEDIFF()** – para calcular a idade ou o tempo entre duas datas;
 - • **DAYNAME()** – para descobrir o dia da semana de uma determinada data.

Relembrando ...

- **Aritméticas**, como somar, subtrair ou calcular porcentagens diretamente nas colunas (ex: calcular salário com bônus ou total de vendas).
- **Funções de texto**, para juntar nomes, converter para maiúsculas ou minúsculas, extrair partes de palavras, etc.
- **Funções de data**, para formatar datas, calcular idades ou descobrir o dia da semana de um evento.
- **Expressões de comparação**, como $>$, $<$, $=$, $\neq$, que usamos nos filtros (**WHERE**) ou nas ordenações (**ORDER BY**).
 - **Expressões de comparação**, como $>$, $<$, $=$, $\neq$, que usamos para filtrar (**WHERE**) ou ordenar os dados (**ORDER BY**).

O que é o **CASE**?

- O **CASE** é uma expressão que permite **mostrar diferentes resultados dependendo de uma condição**.
- O **CASE** funciona como uma estrutura condicional dentro do SQL, semelhante ao "if" em linguagens de programação.



INSTITUTO
FEDERAL
Santa Catarina

Ideias de situações em que podemos aplicar o **CASE**?

- **Classificar** registros em categorias (ex: nota → aprovado/reprovado);
- **Traduzir valores** para algo mais legível (ex: código 1 vira “Sim”, código 0 vira “Não”);
- **Aplicar lógicas condicionais** dentro da própria consulta, de forma simples e clara.

Como funciona o **CASE** no MySQL?

```
CASE  
  WHEN condição1 THEN resultado1  
  WHEN condição2 THEN resultado2  
  ELSE resultado_padrao  
END
```

CASE

WHEN condição1 THEN resultado1

WHEN condição2 THEN resultado2

ELSE resultado_padrao

END

- **O CASE começa com a palavra-chave CASE** e pode ter várias verificações dentro dele.
- **Cada WHEN testa uma condição** e define um resultado para aquela situação.
- **O ELSE é opcional** – Se omitido e nenhuma condição for atendida, o retorno será NULL.
- **O CASE sempre finaliza com END**, garantindo que ele seja interpretado corretamente pelo SQL.
- O **CASE** não altera a estrutura da tabela original, ele apenas classifica os dados na consulta **temporariamente**.

Exemplo de CASE Simples

```
SELECT idEntrega, endereco, peso,  
CASE  
    WHEN peso <= 10 THEN 'Leve'  
    WHEN peso BETWEEN 11 AND 50 THEN 'Médio'  
    ELSE 'Pesado'  
END AS CategoriaPeso  
FROM Entregas;
```

- Se o peso for menor ou igual a 10, a entrega é categorizada como "**Leve**".
- Se o peso estiver entre 11 e 50, a entrega é categorizada como "**Médio**".
- Se o peso for maior que 50, a entrega é categorizada como "**Pesado**".

Exemplo de **CASE** Simples

idEntrega	Endereço	Peso	CategoriaPeso
1	Rua A, 123	8.5	Leve
2	Rua B, 456	25	Médio
3	Rua C, 789	70	Pesado

Exemplo de **CASE** Busca

```
SELECT nome,salario  
CASE salario  
    WHEN 1000 THEN 'Baixo'  
    WHEN 5000 THEN 'Médio'  
    WHEN 10000 THEN 'Alto'  
    ELSE 'Elevado'  
END AS FaixaSalarial  
FROM Funcionarios;
```

- ◆ O **CASE** compara diretamente o salário com valores específicos.
- ◆ Se o salário for **1000**, retorna "**Baixo**".
- ◆ Se for **5000**, retorna "**Médio**".



INSTITUTO
FEDERAL
Santa Catarina

Exemplo de CASE Busca

Nome	Salário	FaixaSalarial
João	1000	Baixo
Maria	5000	Médio
Pedro	10000	Alto
Fernanda	12000	Pesado

Classificação de Entregas

Objetivo: Nesta atividade, você deverá utilizar **CASE** para categorizar entregas conforme a origem e o tipo de transporte utilizado.

Contexto: Uma empresa de logística precisa classificar suas entregas de acordo com o estado de destino e o modo de transporte. O banco de dados possui uma tabela chamada **Entregas**, onde os registros contêm informações como endereço, cidade, UF, CEP e peso da carga.

Classificação de Entregas

A consulta SQL utilizará **CASE** para definir duas classificações: **Tipo de entrega** → Se for dentro de SC, será "**Estadual**", senão será "**Interestadual**".

Modo de transporte → Com base no peso da carga e na UF:

- Se o peso for **maior ou igual a 200 kg**, o transporte será "**terrestre**".
- Se o peso for **menor que 200 kg e o estado for SC**, o transporte será "**terrestre**".
- Caso contrário, o transporte será "**aéreo**".

-- Criação do banco de dados

```
CREATE DATABASE TRANSPORTE;  
USE TRANSPORTE;
```

-- Criação da tabela

```
CREATE TABLE Entregas (  
  idEntrega INT AUTO_INCREMENT PRIMARY KEY,  
  endereco TEXT NOT NULL,  
  cidade TEXT NOT NULL,  
  UF TEXT NOT NULL,  
  CEP TEXT NOT NULL,  
  peso DOUBLE NOT NULL,  
);
```

INSERT INTO Entregas (endereco, cidade, UF, CEP, peso)

VALUES

('Rua ABC, 123', 'Sao Jose', 'SC', '88000-000', 10.1),
('Rua HIJ, 789', 'Sao Paulo', 'SP', '19000-000', 30),
('Rua DEF, 456', 'Belo Horizonte', 'MG', '30672-772', 200.0);

Exemplo

```
SELECT idEntrega, endereco, cidade, uf, CEP, peso,  
CASE UF  
    WHEN 'SC' THEN 'Estadual'  
    ELSE 'Interestadual'  
END Tipo,  
  
CASE  
    WHEN peso >= 200 THEN 'transporte terrestre'  
    WHEN peso < 200 AND UF = 'SC' THEN 'transporte terrestre'  
    ELSE 'transporte aéreo'  
END Transporte  
FROM Entregas;
```

Classificação de Notas dos Alunos

idEntrega	Endereço	Cidade	UF	Peso	Tipo	Transporte
1	Rua ABC, 123	São José	SC	10.1	Estadual	Transporte terrestre
2	Rua HIJ, 789	São Paulo	SP	30	Interestadual	Transporte aéreo
3	Rua DEF, 456	Belo Horizonte	MG	200	Interestadual	Transporte terrestre



INSTITUTO
FEDERAL
Santa Catarina

Validação de dados diretamente no banco

Restrição **CHECK** no SQL

CHECK é utilizada para **validar valores**, garantindo que apenas dados corretos sejam inseridos.



INSTITUTO
FEDERAL
Santa Catarina

Relembrando . . .

Qual é a diferença entre **NOT NULL** e **UNIQUE**?

Relembrando . . .

Qual é a diferença entre **NOT NULL** e **UNIQUE**?

Ao definir uma coluna como **NOT NULL**, garantimos que nenhum valor **nulo** possa ser inserido.

Já a restrição **UNIQUE** impede que valores duplicados sejam armazenados, mas permite valores nulos, a menos que seja usada em conjunto com **NOT NULL**.

CHECK

CHECK é utilizada para **validar valores**, garantindo que apenas dados corretos sejam inseridos.

A restrição **CHECK** permite definir **regras lógicas** para validar valores dentro de uma coluna. Ela pode usar comparações matemáticas e condicionais para garantir a integridade dos dados.

- Melhora a **integridade** dos dados.
 - A integridade dos dados garante que as informações armazenadas sejam corretas, consistentes e válidas.

- Melhora a **integridade** dos dados.
- Impede inserções ou atualizações **inválidas**.
 - Toda vez que alguém tentar fazer um **INSERT** ou **UPDATE**, o banco **verifica se a condição CHECK é satisfeita**. Se não for, ele dá erro e cancela a operação.

- Melhora a **integridade** dos dados.
- Impede inserções ou atualizações **inválidas**.
- Reduz a complexidade da aplicação.
 - Não precisa programar essas validações do lado da aplicação (em Python, Java, PHP, etc.), porque o **banco já faz isso**

CHECK

- Melhora a **integridade** dos dados.
- Impede inserções ou atualizações **inválidas**.
- Reduz a complexidade da aplicação.

- Aumenta a **segurança** dos dados.
 - Evita que usuários mal-intencionados (ou distraídos) insiram dados errados no banco.

CHECK

- Melhora a **integridade** dos dados.
- Impede inserções ou atualizações **inválidas**.
- Reduz a complexidade da aplicação.
- Aumenta a **segurança** dos dados.

CHECK nível coluna?

```
CREATE TABLE nome_da_tabela (  
  nome_coluna1 TIPO_DADO,  
  nome_coluna2 TIPO_DADO CHECK (condição_lógica),  
  nome_coluna3 TIPO_DADO  
);
```

- nome_coluna2 TIPO_DADO CHECK (condição_lógica)
 - Aqui o CHECK está **diretamente associado a essa coluna**.
 - A condição é **avaliada apenas sobre o valor dessa coluna**.
 - Por exemplo, a expressão pode ser algo como CHECK (nome_coluna2 > 0).
- Quando a restrição se aplica somente a uma coluna específica.



CHECK nível tabela

```
CREATE TABLE nome_da_tabela (  
    nome_coluna1 TIPO_DADO,  
    nome_coluna2 TIPO_DADO,  
    nome_coluna3 TIPO_DADO,  
    CHECK (condição_lógica));
```

- **CHECK (condição_lógica)**
 - Aqui o **CHECK** é **declarado fora das colunas**, mas **ainda dentro da definição da tabela**.
 - Isso permite **verificar condições que envolvam uma ou mais colunas ao mesmo tempo**.
 - Exemplo: **CHECK (nome_coluna2 > 0 AND nome_coluna3 IS NOT NULL)**
- Validando **vários campos em conjunto**.

Exemplo **CHECK** coluna

```
CREATE TABLE Pessoa (  
  idPessoa INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(255) NOT NULL,  
  sobrenome VARCHAR(255) NOT NULL,  
  email VARCHAR(255) NOT NULL,  
  telefone VARCHAR(20) NOT NULL CHECK (CHAR_LENGTH(telefone) >= 14)  
);
```

- **CHECK (CHAR_LENGTH(telefone) >= 14)**: Essa restrição **impede** o **INSERT** ou **UPDATE** se o número de caracteres no campo **telefone** for **menor que 14**.
- O **CHAR_LENGTH** mede **quantos caracteres o texto tem**, incluindo parênteses, espaços e hífen.

Exemplo **CHECK** coluna

```
INSERT INTO Pessoa (nome, sobrenome, email, telefone)
VALUES ('Juca', 'Pereira', 'juca@email.com', '3381-2100');
```

Exemplo **CHECK** coluna

```
INSERT INTO Pessoa (nome, sobrenome, email, telefone)
VALUES ('Juca', 'Pereira', 'juca@email.com', '3381-2100');
```

- **Erro :** Porque '3381-2100' tem **apenas 9 caracteres**. A condição `CHAR_LENGTH(telefone) >= 14` **não é satisfeita**.

Exemplo **CHECK** coluna

```
INSERT INTO Pessoa (nome, sobrenome, email, telefone)
VALUES ('Juca', 'Pereira', 'juca@email.com', '(48) 3381-2100');
```

Exemplo **CHECK** coluna

```
INSERT INTO Pessoa (nome, sobrenome, email, telefone)
VALUES ('Juca', 'Pereira', 'juca@email.com', '(48) 3381-2100');
```

- **Sucesso:** porque o telefone tem **15 caracteres**, e satisfaz a condição.



Exemplo CHECK tabela

```
CREATE TABLE Produtos (  
  idProduto INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(255) NOT NULL,  
  valor DECIMAL(10,2) NOT NULL,  
  desconto DECIMAL(10,2) NOT NULL DEFAULT 0,  
  CHECK (valor >= desconto AND desconto >= 0 AND valor >= 0));
```

- CHECK (valor >= desconto AND desconto >= 0 AND valor >= 0)
- **Restrição múltipla** que impõe 3 condições:
 - O valor **não pode ser menor que o desconto.**
 - O desconto **não pode ser negativo.**
 - O valor do produto **também não pode ser negativo**

Exemplo **CHECK** tabela

```
INSERT INTO Produtos (nome, valor, desconto)
VALUES ('Caneta', 5.00, 6.00);
```


Exemplo **CHECK** tabela

```
INSERT INTO Produtos (nome, valor, desconto)
VALUES ('Caneta', 5.00, 6.00);
```

- **Erro:** a condição **valor $\geq$ desconto** não é satisfeita, pois $5 < 6$.

Check a tabela Alunos

Crie uma tabela chamada **Alunos** com os seguintes requisitos:

1. **Idade mínima de 16 anos.**
2. **Nota final entre 0 e 10 (valores inteiros).**
3. **Frequência entre 0% e 100% (valores inteiros).**
4. A aprovação deve acontecer se o aluno **tiver nota maior ou igual a 7 ou frequência maior ou igual a 75%.**

Check a tabela Alunos

Use restrições **CHECK**:

- No nível de coluna: validações simples como intervalo permitido.
- No nível da tabela: regra de aprovação combinada

```
CREATE TABLE Alunos (  
  idAluno INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  idade INT CHECK (idade >= 16), -- restrição de coluna  
  notaFinal INT CHECK (notaFinal BETWEEN 0 AND 10), -- restrição de coluna  
  frequencia INT CHECK (frequencia BETWEEN 0 AND 100), -- restrição de coluna  
  
  CHECK (notaFinal >= 7 OR frequencia >= 75) -- restrição de tabela  
);
```



INSTITUTO
FEDERAL
Santa Catarina

Exemplo

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)  
VALUES ('Renata', 21, 10, 95);
```

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)
VALUES ('Renata', 21, 10, 95);
```

- **Sucesso:** frequência e nota alta

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)
VALUES ('Ana', 18, 12, 90);
```

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)
VALUES ('Ana', 18, 12, 90);
```

- **Erro :** nota fora do intervalo permitido.

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)
VALUES ('João', 18, 5, 60);
```

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)
VALUES ('João', 18, 5, 60);
```

- **Erro :** nota baixa e frequência baixa.

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)
VALUES ('Carlos', 15, 9, 80);
```

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)
VALUES ('Carlos', 15, 9, 80);
```

- **Erro:** idade menor que 16.

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)
VALUES ('Laura', 18, 9, 60);
```

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)
VALUES ('Laura', 18, 9, 60);
```

- **Sucesso:** nota suficiente, frequência baixa

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)
VALUES ('Bruno', 20, 6, 80);
```

```
INSERT INTO Alunos (nome, idade, notaFinal, frequencia)
VALUES ('Bruno', 20, 6, 80);
```

- **Sucesso:** frequência suficiente, nota baixa

CHECK + CASE

- Posso usar **CASE** dentro do **CHECK**?
 - Tecnicamente, não. O **CHECK** aceita expressões booleanas, não aceita **CASE**
- Como usar lógica do **CASE** dentro de **CHECK**?
 - Pode reescrever a lógica condicional de um **CASE** em forma booleana, o que pode **simular** o comportamento de um **CASE**.

CHECK + CASE

Se o tipo do produto for 'eletrônico', o valor deve ser maior que 100.
Para os demais tipos, o valor pode ser qualquer um acima de 0.

```
CASE  
  WHEN tipo = 'eletrônico' AND valor > 100 THEN 'ok'  
  WHEN tipo != 'eletrônico' AND valor > 0 THEN 'ok'  
  ELSE 'erro'  
END
```



CASE

WHEN tipo = 'eletrônico' AND valor > 100 **THEN** 'ok'

WHEN tipo != 'eletrônico' AND valor > 0 **THEN** 'ok'

ELSE 'erro'

END

equivalente

CHECK (

(tipo = 'eletrônico' AND valor > 100)

OR

(tipo != 'eletrônico' AND valor > 0)

)

Criar uma tabela **Aluno** que atenda às seguintes regras com base no **sexo biológico** informado no momento do cadastro:

- O campo **sexo biológico** deve aceitar apenas os valores 'M' (masculino) e 'F' (feminino).
- Se o **sexo biológico** for 'M', o campo **reservista** deve ser preenchido.
- Se o **sexo biológico** for 'F', o campo **reservista** pode ser nulo.

A validação deve ser feita diretamente no **Sistema Gerenciador de Banco de Dados (SGBD)**, usando **restrições CHECK**.

```
CREATE TABLE Aluno (  
  idAluno INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(255) NOT NULL,  
  email VARCHAR(255) NOT NULL,  
  sexobiologico ENUM('M', 'F') NOT NULL,  
  reservista VARCHAR(20),
```

-- Garante que o sexo biológico seja M ou F (redundância útil)

```
CHECK (sexobiologico IN ('M', 'F')),
```

-- Validação condicional: se for sexo biológico masculino, reservista é obrigatório

```
CHECK ((sexobiologico = 'M' AND reservista IS NOT NULL) OR sexobiologico = 'F')
```

```
);
```

```
INSERT INTO Aluno (nome, email, sexobiologico, reservista)  
VALUES ('Juca', 'juca@email.com', 'X', '13345');
```

- **Erro:** sexo inválido (não é 'M' nem 'F')

```
INSERT INTO Aluno (nome, email, sexobiologico)
VALUES ('Juca', 'juca@email.com', 'M');
```

- **Erro:** sexo masculino sem número de reservista

```
INSERT INTO Aluno (nome, email, sexobiologico, reservista)
```

```
VALUES ('Juca', 'juca@email.com', 'M', '1234');
```

- **Sucesso:** sexo masculino com número de reservista


```
INSERT INTO Aluno (nome, email, sexobiologico)
VALUES ('Ana', 'ana@email.com', 'F');
```

- **Sucesso:** sexo feminino, reservista pode ser nulo

Uma **transação** representa um bloco de operações SQL executadas de forma **atômica**.

O banco garante que:

- Se tudo correr bem, as alterações são **persistidas**.
- Se ocorrer um erro, as alterações são **desfeitas**.

Transações

Em sistemas de bancos de dados transacionais, toda execução de uma instrução SQL ocorre dentro de uma transação. Assim que a instrução termina, sua alteração é automaticamente persistida.

Transações

Em sistemas de bancos de dados transacionais, toda execução de uma instrução SQL ocorre dentro de uma transação. Assim que a instrução termina, sua alteração é automaticamente persistida.

- Em bancos de dados relacionais, **toda instrução SQL** é automaticamente uma transação.
- A instrução é **confirmada** (commit) ao terminar, **sem intervenção do usuário**.

Controle Manual de Transações

Gestão Financeira

- Um banco processa transferências de dinheiro de forma transacional.

Compra Online

- Um usuário escolhe produtos, adiciona ao carrinho e vai para o pagamento.

Controle Manual de Transações

- **BEGIN**: Inicia uma transação manual.
- **COMMIT**: Confirma e persiste todas as alterações feitas dentro da transação.
- **ROLLBACK**: Desfaz todas as alterações realizadas dentro da transação.
- **END TRANSACTION** é um **sinônimo de COMMIT**.
 - Útil para quem deseja organizar o código de forma mais clara ou semântica.

Exemplo

```
CREATE TABLE Contas (  
    numeroConta INTEGER NOT NULL PRIMARY KEY,  
    saldo DECIMAL NOT NULL DEFAULT 0,  
    CHECK(saldo >= 0) );
```

```
CREATE TABLE historico_operacoes (  
    numeroOperacao INTEGER PRIMARY KEY AUTOINCREMENT,  
    numeroConta INTEGER NOT NULL,  
    operacao TEXT NOT NULL,  
    total DECIMAL NOT NULL,  
    data TEXT NOT NULL,  
    CHECK(operacao in ('+', '-')),  
    FOREIGN KEY (numeroConta) REFERENCES Contas(numeroConta)  
);
```

```
INSERT INTO Contas VALUES (123,1000),(456,50);
```

Exemplo



INSTITUT
FEDERAL
Santa Catarina

-- Transferindo valores entre contas

BEGIN TRANSACTION;

-- Debitando R\$ 200,00 da conta 123

UPDATE Contas **SET** saldo = saldo - 200 **WHERE** numeroConta = 123;

-- Creditando R\$ 200,00 na conta 456

UPDATE Contas **SET** saldo = saldo + 200 **WHERE** numeroConta = 456;

-- Registrando a transferência no histórico

INSERT INTO historico_operacoes (numeroConta, operacao, total, data)
VALUES (123,'-',200,datetime('now'))

INSERT INTO historico_operacoes (numeroConta, operacao, total, data)
VALUES (456,'+',200,datetime('now'))

COMMIT;

Transações Aninhadas

Dentro de uma mesma transação, podemos definir pontos intermediários usando:

- **SAVEPOINT**: Marca um ponto específico dentro da transação para possível reversão.
- **RELEASE**: Remove um savepoint, consolidando as alterações feitas antes dele.

Transações Aninhadas

SAVEPOINT → CHECKPOINT Quando você alcança um ponto importante no jogo, ele cria um **checkpoint** (**SAVEPOINT**). Se algo der errado, pode voltar para esse momento, sem precisar recomeçar do início.

- **ROLLBACK TO → VOLTAR PARA UM CHECKPOINT** Digamos que você tomou uma decisão ruim no jogo e perdeu muitos pontos. Em vez de reiniciar a fase inteira, você pode **voltar para o último checkpoint** (**ROLLBACK TO savepoint**), desfazendo apenas o que veio depois.
- **RELEASE → APAGAR UM CHECKPOINT SEM VOLTAR** Agora, imagine que você já passou várias partes do jogo e tem certeza de que está indo bem. Você pode remover aquele checkpoint (**RELEASE savepoint**), consolidando as ações anteriores. O jogo continua normalmente, mas sem possibilidade de voltar para aquele ponto.

-- Iniciar a transação

```
BEGIN TRANSACTION;
```

-- Inserir um novo cliente

```
INSERT INTO clientes (id, nome)  
VALUES (1, 'Ana Silva');
```

-- Criar um ponto de salvamento

```
SAVEPOINT save1;
```

-- Inserir um pedido associado ao cliente

```
INSERT INTO pedidos (id, cliente_id, total)  
VALUES (1, 1, 150.00);
```

-- Criar outro ponto de salvamento

```
SAVEPOINT save2;
```

-- Simular um erro ao tentar inserir um produto no pedido

```
INSERT INTO produtos_pedido (pedido_id, produto_id, quantidade)  
VALUES (1, 999, 2);
```

-- Esse produto_id 999 não existe

-- Reverter apenas até o último savepoint (desfazendo o produto, mas mantendo o pedido)

```
ROLLBACK TO save2;
```

-- Liberar o primeiro ponto de salvamento

```
RELEASE save1;
```

-- Finalizar a transação

```
COMMIT;
```



```
SELECT * FROM historico_operacoes;
```

```
BEGIN TRANSACTION;
```

-- Resultará em erro, pois não tem saldo suficiente

```
UPDATE Contas SET saldo = saldo - 4000 WHERE numeroConta = 123;
```

-- Será registrado normalmente

```
INSERT INTO historico_operacoes (numeroConta, operacao, total, data)  
VALUES (123,'-',4000,datetime('now'));
```

```
SELECT * FROM historico_operacoes;
```

-- Irá desfazer tudo que foi alterado nesta transação

```
ROLLBACK;
```

```
SELECT * FROM historico_operacoes;
```

Nota de Referência:
O material apresentado tem como base os conteúdos elaborados
pelo **Professor Emerson Ribeiro de Mello**